UNIVERSIDADE FEDERAL DE SERGIPE

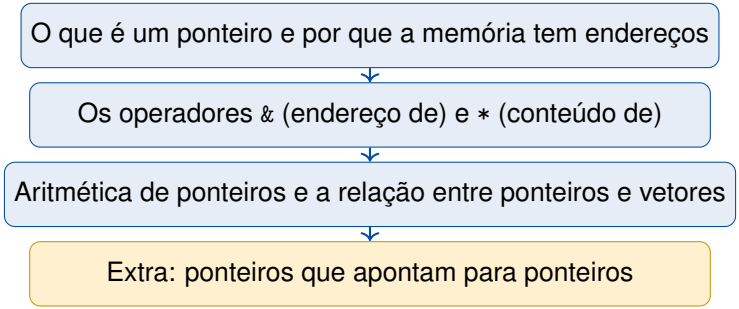
Ponteiros em C

Conceitos, operadores e aritmética de ponteiros

Prof. André Yoshiaki Kashiwabara

DCOMP – Universidade Federal de Sergipe
andreyoshiaki@dcomp.ufs.br

O que você vai aprender hoje



Onde isso entra na disciplina

Ponteiros são a base do que vem a seguir: alocação dinâmica, listas, pilhas, filas e arquivos.

O que é um ponteiro

Por que precisamos de ponteiros?

Problema:

Uma função em C recebe **cópias** dos argumentos. Como escrever uma função que *altera* as variáveis de quem a chamou? Como devolver mais de um resultado? Como passar um vetor de um milhão de elementos sem copiar um milhão de elementos?

Solução:

Em vez de passar o *valor*, passamos o *endereço* onde o valor mora. Com o endereço em mãos, a função chega até a variável original e escreve nela. Esse endereço guardado em uma variável é o que chamamos de **ponteiro**.

Você já usou um

O `&` do `scanf("%d", &x)` da primeira aula era exatamente isso: o endereço de `x`.

O programa de hoje: `troca_soma.c`

```c
#include <stdio.h>

void troca_v1(int a, int b) {
    int t = a;
    a = b;
    b = t;
}
void troca_v2(int *a, int *b) {
    int t = *a;
    *a = *b;
    *b = t;
}
```

```c
int main(void) {
    int x = 10, y = 20;
    troca_v1(x, y);
    printf("v1: x=%d y=%d\n", x, y);
    troca_v2(&x, &y);
    printf("v2: x=%d y=%d\n", x, y);
    int v[5] = {2, 4, 6, 8, 10};
    int *p = v;
    printf("v0=%d *p=%d p2=%d\n",
           v[0], *p, *(p + 2));
    int soma = 0;
    for (int *q = v; q < v + 5; q++)
        soma += *q;
    printf("soma=%d\n", soma);
    return 0;
}
```

O que este programa imprime?

Em duplas, anotem a previsão (3–4 min) — sem computador!

1. Depois de `troca_v1(x, y)`, quanto valem `x` e `y`?
2. E depois de `troca_v2(&x, &y)`?
3. Quanto vale `*p`? E `*(p + 2)`?
4. Qual é o valor final de `soma`?

Regra do jogo

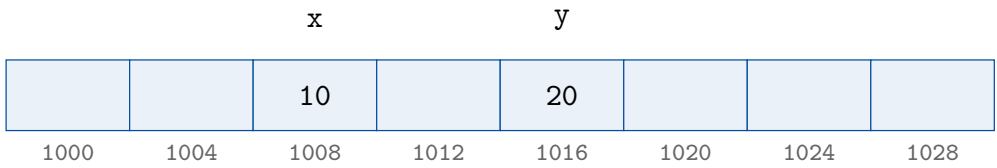
Escreva as quatro linhas de saída exatamente como você espera vê-las. Previsão errada não é problema: é o ponto de partida da investigação.

Compile, execute e compare

```
$ gcc troca_soma.c -o troca_soma
$ ./troca_soma
v1: x=10 y=20
v2: x=20 y=10
v0=2 *p=2 p2=6
soma=30
```

- As duas funções parecem fazer a mesma coisa — mas só uma delas trocou.
- `*(p + 2)` deu 6, e não 4: por que o “+2” pulou dois *inteiros*, e não dois *bytes*?
- Onde sua previsão divergiu? É exatamente aí que mora a aula de hoje.

A memória é uma fila de bytes numerados



Ideia central

Toda variável ocupa um pedaço da memória, e esse pedaço tem um **endereço**: um número que identifica sua posição. Declarar `int x = 10;` reserva 4 bytes e dá um nome a eles.

Definição: ponteiro

Definição

Um **ponteiro** é uma variável cujo valor é o *endereço* de outra variável. Um ponteiro tem tipo: `int *p` guarda o endereço de um `int`; `double *q` guarda o endereço de um `double`.

Intuição

O ponteiro é o número do apartamento, não o morador. Com o número em mãos, você chega ao apartamento e vê (ou troca) quem mora lá. Duas pessoas com o mesmo número chegam ao mesmo apartamento — por isso uma função com o endereço consegue alterar a variável original.

Declarando ponteiros

```
1  int  x = 10;       /* x guarda um inteiro                          */
2  int *p;            /* p guarda o endereco de um inteiro            */
3  p = &x;            /* agora p aponta para x                        */
4
5  double  d = 3.14;
6  double *pd = &d;   /* declaracao e inicializacao juntas            */
7
8  int *a, b;         /* CUIDADO: a e ponteiro, b e int comum         */
```

- O $*$ pertence à *variável*, não ao tipo: por isso `int *a, b;` declara um ponteiro e um inteiro.
- O tipo do ponteiro importa: ele diz ao compilador quantos bytes ler e quanto avançar na aritmética.
- Todo ponteiro ocupa o mesmo tamanho (8 bytes em máquinas de 64 bits).

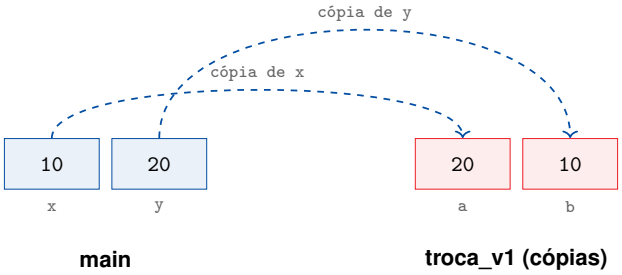
Recapitulando: o conceito

O que aprendemos

- Cada variável ocupa bytes da memória e tem um endereço.
- Um ponteiro é uma variável que guarda um endereço.
- O tipo do ponteiro descreve o que existe naquele endereço.
- Passar endereços permite que uma função altere variáveis de quem a chamou.

Os operadores & e *

Por que `troca_v1` não trocou nada?



Passagem por valor

Em C, argumentos são sempre copiados: a troca aconteceu nas cópias `a` e `b`, e `x` e `y` nem ficaram sabendo.

Os dois operadores fundamentais

Operador	Lê-se	Resultado
`&x`	“endereço de `x`”	o endereço onde `x` está
`*p`	“conteúdo apontado por `p`”	o valor guardado naquele endereço

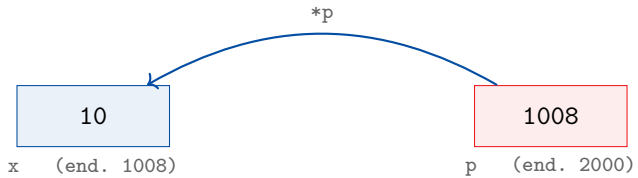
São operações inversas

`*(&x)` $\equiv$ `x` e se `p = &x`, então `*p` $\equiv$ `x`

Atenção ao contexto

Na *declaração* `int *p;` o `*` anuncia “`p` é um ponteiro”. No *uso* `*p = 7;` o `*` significa “vá até lá e escreva”.

Um retrato da memória




```
int x = 10;
int *p = &x;
```

`p` vale 1008
`*p` vale 10
`&p` vale 2000

Leia sempre nesta ordem

`p` é um endereço; `*p` é o valor que mora nele; `&p` é o endereço do próprio ponteiro.

Vendo os endereços com os próprios olhos

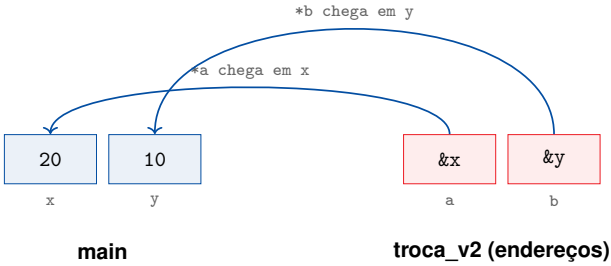
```c
int  x = 10;
int *p = &x;

printf("valor de x  : %d\n",  x);     /* 10                */
printf("endereco de x: %p\n", (void *) &x);
printf("valor de p  : %p\n",  (void *) p);   /* igual a &x */
printf("conteudo *p : %d\n",  *p);    /* 10                */

*p = 99;                              /* escreve em x       */
printf("agora x vale %d\n", x);       /* 99                */
```

- `%p` imprime endereços; o `(void *)` é a conversão exigida pelo padrão.
- Os endereços mudam a cada execução — o que importa é a *relação* entre eles, não o número.

Por que `troca_v2` funcionou



Regra prática

As cópias agora são dos *endereços*: `*a` e `*b` alcançam `x` e `y` originais. Para alterar uma variável do chamador, a função recebe `&variavel` e usa `*parametro`.

Devolvendo mais de um resultado

```c
void min_max(int v[], int n, int *pmin, int *pmax) {
    *pmin = *pmax = v[0];
    for (int i = 1; i < n; i++) {
        if (v[i] < *pmin) *pmin = v[i];
        if (v[i] > *pmax) *pmax = v[i];
    }
}

int menor, maior;
min_max(v, 5, &menor, &maior);   /* dois resultados de uma vez */
```

- `return` devolve um valor só; ponteiros devolvem quantos você quiser.
- É o mesmo mecanismo do `scanf`: ele preenche a variável cujo endereço você entregou.

Armadilhas com & e *

```c
int *p;
*p = 5;             /* ERRO: p nao aponta para lugar nenhum */

int *q = NULL;      /* ponteiro nulo: "nao aponta para nada" */
if (q != NULL)      /* sempre teste antes de usar            */
    printf("%d\n", *q);

printf("%d\n", q); /* ERRO: %d espera int, nao endereco    */
```

Três regras de sobrevivência

1. Inicialize sempre — com `&algo` ou com `NULL`.
2. Nunca faça `*p` com `p` nulo ou inválido.
3. Compile com `-Wall` e leia os avisos.

Recapitulando: os operadores

O que aprendemos

- `&x` produz o endereço de `x`; `*p` acessa o conteúdo apontado por `p`.
- `*` e `&` são inversos: `*(&x)` é `x`.
- Passar `&x` deixa a função alterar `x` — é assim que `scanf` e `troca_v2` funcionam.
- `NULL` marca “não aponta para nada”; teste antes de desreferenciar.

Aritmética de ponteiros

Por que `*(p + 2)` deu 6, e não 4?

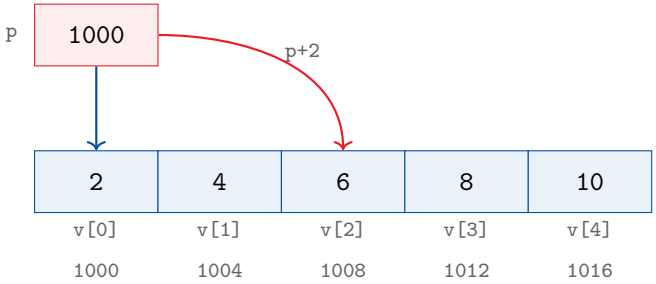
A pergunta

Se `p` guarda um endereço — um número — então `p + 2` deveria ser “dois a mais”. Dois o quê? Dois bytes? Dois inteiros?

Anote sua hipótese antes de virar o slide

- Se `p` vale 1000 e `p` é `int *`, quanto vale `p + 2`?
- E se `p` fosse `double *`?
- E se fosse `char *`?

A resposta: a aritmética conta elementos, não bytes



Regra

$$\texttt{p + k} = \text{endereço de } \texttt{p} + k \times \texttt{sizeof(tipo apontado)}$$

Como `sizeof(int)` vale 4, `p + 2` avança 8 bytes: de 1000 para 1008, onde mora o 6.

Vetor e ponteiro: o mesmo endereço

```c
int v[5] = {2, 4, 6, 8, 10};
int *p = v;         /* o nome do vetor vale o endereco de v[0] */
```

Escrita com índice	Escrita com ponteiro	Endereço	Valor
`v[0]`	`*v`	`v`	2
`v[2]`	`*(v + 2)`	`v + 2`	6
`p[3]`	`*(p + 3)`	`p + 3`	8
`&v[4]`	`v + 4`	—	endereço do último

Definição de `[]`

Em C, `a[i]` é apenas outra forma de escrever `*(a + i)`. Colchete é açúcar sintático para aritmética de ponteiros.

Percorrendo um vetor com ponteiro

Com índice

```
1  int soma = 0;
2  for (int i = 0; i < 5; i++)
3      soma += v[i];
```

Com ponteiro

```
1  int soma = 0;
2  for (int *q = v; q < v + 5; q++)
3      soma += *q;
```

- `q++` avança um `int` (4 bytes), não um byte.
- `v + 5` é o endereço *logo após* o último elemento: pode ser calculado e comparado, mas nunca desreferenciado.
- As duas versões produzem o mesmo `soma = 30`.

O que se pode (e o que não se pode) fazer

Operação	Exemplo	Resultado
ponteiro + inteiro	`p + 3`	ponteiro 3 elementos adiante
ponteiro − inteiro	`p - 1`	ponteiro 1 elemento atrás
incremento/decremento	`p++`, `p-`	anda um elemento
ponteiro − ponteiro	`&v[4] - v`	4 (quantos elementos separam)
comparação	`p < v + 5`	1 ou 0
ponteiro + ponteiro	`p + q`	**não existe**
multiplicar/dividir	`p * 2`	**não existe**

Só faz sentido dentro do mesmo vetor

Comparar ou subtrair ponteiros de vetores diferentes é comportamento indefinido — o programa pode "funcionar" hoje e falhar amanhã.

Exemplo: contando com a diferença de ponteiros

```
1  int *maior(int *v, int n) {
2      int *m = v;                      /* candidato inicial */
3      for (int *q = v + 1; q < v + n; q++)
4          if (*q > *m) m = q;
5      return m;                        /* devolve um ENDERECO */
6  }
7
8  int v[5] = {2, 9, 6, 8, 10};
9  int *m = maior(v, 5);
10 printf("maior = %d na posicao %ld\n", *m, m - v);  /* 10, posicao 4 */
```

- A função devolve *onde* está o maior, não só quanto ele vale.
- `m - v` converte o endereço de volta em índice.

Armadilha: vetor não é ponteiro

```c
int v[5];
int *p = v;

sizeof(v);   /* 20 = 5 * sizeof(int)  -- o vetor inteiro */
sizeof(p);   /*  8 = tamanho de um endereco               */

void f(int v[]) {      /* o parametro E um ponteiro:     */
    sizeof(v);         /* 8, e nao 20!                   */
}
```

Consequência prática

Ao passar um vetor para uma função o tamanho **se perde** — por isso toda função que recebe vetor recebe também o `n`.

Recapitulando: aritmética

O que aprendemos

- `p + k` avança $k \times$ `sizeof(tipo)` bytes.
- `a[i]` é exatamente `*(a + i)`.
- O nome do vetor vale o endereço do primeiro elemento.
- Subtrair ponteiros do mesmo vetor dá a distância em elementos.
- `sizeof` distingue vetor de ponteiro — e o tamanho se perde na passagem para funções.

Extra: ponteiros para ponteiros

Extra: e se o valor guardado for um endereço?

```c
int   x  = 10;
int  *p  = &x;      /* p  guarda o endereco de x */
int **pp = &p;      /* pp guarda o endereco de p */

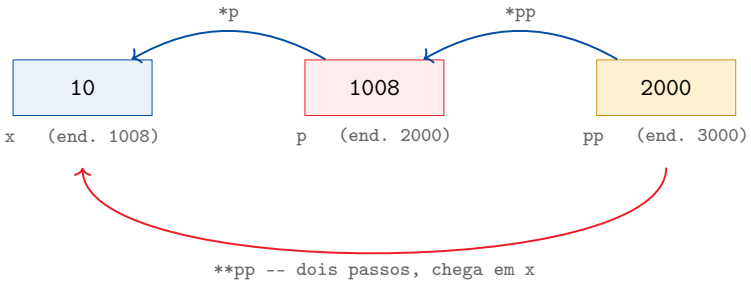
printf("%d\n", x);       /* 10 */
printf("%d\n", *p);      /* 10 */
printf("%d\n", **pp);    /* ?  */

**pp = 99;
printf("%d\n", x);       /* ?  */
```

Antes de executar

Um ponteiro também é uma variável: ocupa memória e tem endereço (`&p`). Nada impede que outra variável guarde esse endereço.

Uma cadeia de endereços



Cada $*$ é um passo na seta

`pp` vale 2000; `*pp` vale 1008 (é o próprio `p`); `**pp` vale 10 (é o próprio `x`). Escrever `**pp = 99` altera `x`.

Lendo a declaração e o uso

Expressão	Tipo	Vale
`pp`	`int **` (endereço de um `int *`)	2000
`*pp`	`int *` (endereço de um `int`)	1008, ou seja `p`
`**pp`	`int` (o valor final)	10, ou seja `x`

A regra é a mesma de antes, aplicada duas vezes

Na *declaração*, cada `*` anuncia um nível de indireção: `int **pp` lê-se "`pp` aponta para algo que aponta para `int`". No *uso*, cada `*` desce um nível, até chegar ao valor.

Cuidado

`*pp` não é o valor de `x`: é um endereço. Só o segundo `*` chega ao número.

Para que isso serve na prática

```c
void aponta_maior(int *v, int n, int **saida) {
    int *m = v;
    for (int i = 1; i < n; i++)
        if (v[i] > *m) m = &v[i];
    *saida = m;             /* escreve no ponteiro do chamador */
}

int *achou = NULL;
aponta_maior(v, 5, &achou);
printf("%d na posicao %ld\n", *achou, achou - v);
```

A mesma regra, um nível acima

Para alterar um `int`, a função recebe `int *`; para alterar um `int *`, ela recebe `int **` — como em `main(int argc, char **argv)`.

Mãos à obra

Altere o programa

Tarefa 1 — dobrar sem colchetes

Escreva `void dobra(int *v, int n)` que multiplica cada elemento por 2 usando **apenas** `*` e aritmética de ponteiros — sem `[]` em nenhum lugar da função.

Tarefa 2 — devolver um endereço

Implemente `int *maior(int *v, int n)` e imprima, no `main`, o maior valor *e* sua posição usando `m - v`.

Tarefa 3 — dois resultados

Implemente `void min_max(int *v, int n, int *pmin, int *pmax)` e mostre que ela preenche as duas variáveis do `main` de uma só vez.

Desafio

Inverter um vetor com dois ponteiros

Escreva `void inverte(int *ini, int *fim)` que recebe o endereço do primeiro elemento e o endereço *logo após* o último, e inverte o vetor no lugar — trocando os extremos e caminhando para o centro, sem usar índices e sem vetor auxiliar.

Critérios

- Deve funcionar para vetores de tamanho par e ímpar (teste com 5 e 6).
- Reaproveite `troca_v2` para a troca.
- Chamada esperada: `inverte(v, v + n);`

Fechamento

Ponteiro = variável que guarda um endereço

↓

`&` pega o endereço; `*` chega ao conteúdo

↓

`a[i]` $\equiv$ `*(a + i)`: colchete é aritmética de ponteiros

↓

`int **pp`: cada `*` é mais um passo até o valor

↓

Próxima aula: alocação dinâmica — memória que você mesmo pede

Para casa

O tutorial traz as três tarefas com testes automáticos, o desafio e o extra sobre ponteiros para ponteiros.

Referências I


André Backes.
Linguagem C: completa e descomplicada.
Elsevier, Rio de Janeiro, 2013.


Paul Deitel and Harvey Deitel.
C: como programar.
Pearson Prentice Hall, São Paulo, 6 edition, 2011.


Paulo Feofiloff.
Algoritmos em linguagem C.
Elsevier, Rio de Janeiro, 2009.

Referências II


Brian W. Kernighan and Dennis M. Ritchie.
C: a linguagem de programação — padrão ANSI.
Campus, Rio de Janeiro, 1989.


Herbert Schildt.
C completo e total.
Makron Books, São Paulo, 3 edition, 1997.


Aaron M. Tenenbaum, Yedidyah Langsam, and Moshe J. Augenstein.
Estruturas de dados usando C.
Pearson Makron Books, São Paulo, 1995.